Spatial Databases and PostGIS

Class 2

Coordinate systems, projections, and distance functions in PostGIS

Today

- Geographic vs projected coordinate systems
- WGS84 (EPSG:4326) and Portugal TM06 (EPSG:3763)
- Why SRIDs matter before measuring distance
- Using ST_Transform, ST_Distance and ST_DistanceSpheroid

Key idea

The same feature can have different coordinate values in different CRS.

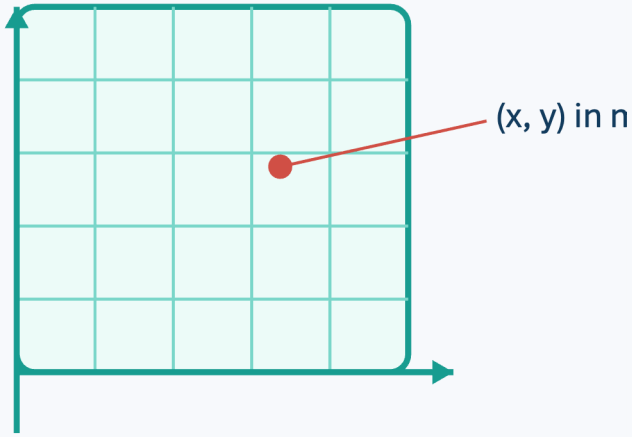
In PostGIS, correct analysis starts by choosing the right SRID and the right distance model.

Geographic vs projected coordinate systems

The first choice affects every measurement that follows

Geographic CRS

angles on the ellipsoid



Projected CRS

Geographic CRS

- Coordinates are angular: longitude and latitude
- Usually stored in degrees
- Natural for GPS and web data exchange

Projected CRS

- Coordinates are planar: x and y
- Units are typically metres
- Better for distance, area and buffering

WGS84 / EPSG:4326

The default CRS behind GPS coordinates and much exchanged web data

What it is

- Global geodetic CRS
- Axis directions: latitude, longitude in the EPSG definition
- Units: degrees
- Very common as the input format of points from apps, GPS and APIs

Typical PostGIS point

```
SELECT ST_SetSRID(
   ST_MakePoint(-9.1393, 38.7223),
   4326
 );
```

Important consequence

If you keep data as geometry in EPSG:4326 and call ST_Distance, the answer is expressed in the spatial reference units: degrees, not metres.

So EPSG:4326 is excellent for storage and interchange, but not the safest direct choice for planar measurement.

ETRS89 / Portugal TM06 - EPSG:3763

A projected CRS in metres for mainland Portugal

Why we use it in class

- Projected coordinate system for Portugal mainland onshore
- Cartesian axes: easting and northing
- Unit: metre
- Good default for local Portuguese analysis: distance, area, buffers and joins

Remember

Projected systems always introduce distortion, but the goal is to keep it small over the intended area of use.

Typical usage pattern

```
-- Load or receive lon/lat data in 4326
-- Transform before metric analysis
SELECT ST_Distance(
  ST_Transform(a.geom, 3763),
  ST_Transform(b.geom, 3763)
)
FROM places a, places b;
```

For mainland Portugal, this is usually the most natural planar workspace in PostGIS.

Do not confuse ST_SetSRID with ST_Transform

One assigns metadata; the other changes coordinates

ST_SetSRID(geom, srid)

Assigns or fixes the SRID label.
It does not move the geometry.

```
SELECT ST_SetSRID(
  ST_MakePoint(-9.1393, 38.7223), 4326
);
```

ST_Transform(geom, srid)

Converts the geometry from one CRS to another.
The coordinate values change.

```
SELECT ST_Transform(
  geom_4326, 3763
);
```

Example: transforming a Portuguese point to EPSG:3763

Bragança shown here just to make the metric coordinates easy to read

```
SELECT ST_AsText(
  ST_Transform(
    ST_SetSRID(ST_MakePoint(-6.7567, 41.8060), 4326),
    3763
  )
);
```

Approximate result

POINT(114383 238310)

Now the coordinates are in metres on a planar system, ready for distance and buffering operations.

What changed?

- The geometry stayed the same real-world location
- Only its coordinate representation changed
- The new values are not degrees anymore
- Metric calculations now make sense in the chosen projected CRS

In practice: receive in 4326, analyse in 3763.

Distance functions: choose the right model

Same places, different answers depending on geometry/geography and CRS

Function	Input pattern	Units	Speed	Use it when
ST_Distance	geometry in EPSG:3763	metres	fast	Most classroom and national-scale metric work in Portugal
ST_Distance	geometry in EPSG:4326	degrees	fast	Usually not what you want for real distance
ST_Distance	geography	metres	slower	Ellipsoidal geodesic distance without manual transformation
ST_DistanceSpheroid	lon/lat geometry + spheroid	metres	slower	Explicit spheroidal distance on geodetic coordinates

Important caveat: ST_DistanceSpheroid ignores the SRID stored in the geometry and assumes the coordinates are based on the spheroid you pass.

Concrete comparison: Lisbon <-> Porto

A useful classroom example showing “degrees vs metres”

```
  WITH pts AS (
    SELECT
      ST_SetSRID(ST_MakePoint(-9.1393, 38.7223), 4326) AS
lisbon,
      ST_SetSRID(ST_MakePoint(-8.6291, 41.1579), 4326) AS
porto
  )
  SELECT
    ST_Distance(lisbon, porto) AS deg_distance,
    ST_Distance(ST_Transform(lisbon, 3763),
                ST_Transform(porto, 3763)) AS planar_m,
   ST_Distance(lisbon::geography, porto::geography) AS
geodesic_m
 FROM pts;
```

Approximate results

ST_Distance(lisbon, porto) = 2.488

ST_Distance(... in 3763) = 273,939 m

ST_Distance(...::geography) = 273,925 m

For Portugal-scale analysis, the projected 3763 result is very close to the geodesic answer, while the raw 4326 geometry answer is clearly not in metres.

Practical rules of thumb for PostGIS work in Portugal

A short checklist students can keep using all semester

Workflow

- Store or receive many point datasets in EPSG:4326
- Transform to EPSG:3763 before measuring distance or area in mainland Portugal
- Use geography when you want ellipsoidal distance directly in metres
- Always check the SRID before trusting a numeric result

Memorise these three lines

```
-- assign an SRID
ST_SetSRID(geom, 4326)

-- change coordinates to a metric CRS
ST_Transform(geom, 3763)

-- then measure in metres
ST_Distance(g1_3763, g2_3763)
```

A result is only as meaningful as the coordinate system behind it.

A Dataset with the Parishes of Portugal

Parishes per region

- Mainland
- Azores oriental and central groups
- Azores ocidental group
- Madeira

- cont_freguesias
- raa_cen_ori_freguesias
- raa_oci_freguesias
- ram_freguesias

```
michelferreira=# \d ram_freguesias;
                                        Table "public.ram_freguesias"
          Column           |             Type              | Collation | Nullable |                    Default
---------------------------+-------------------------------+-----------+----------+-----------------------------------------------
 fid                       | integer                       |           | not null | nextval('ram_freguesias_fid_seq'::regclass)
 geom                      | geometry(MultiPolygon,5016)   |           |          |
 dtmnfr                    | character varying(8)          |           |          |
 freguesia                 | character varying             |           |          |
 municipio                 | character varying             |           |          |
 distrito_ilha             | character varying             |           |          |
 nuts3                     | character varying             |           |          |
 nuts2                     | character varying             |           |          |
 nuts1                     | character varying             |           |          |
 area_ha                   | double precision              |           |          |
 perimetro_km              | integer                       |           |          |
 designacao_simplificada   | character varying             |           |          |
Indexes:
    "ram_freguesias_pkey" PRIMARY KEY, btree (fid)
    "ram_freguesias_geom_geom_idx" gist (geom)
```

```
michelferreira=# \d cont_freguesias
                                        Table "public.cont_freguesias"
          Column           |             Type              | Collation | Nullable |                    Default
---------------------------+-------------------------------+-----------+----------+-----------------------------------------------
 fid                       | integer                       |           | not null | nextval('cont_freguesias_fid_seq'::regclass)
 geom                      | geometry(MultiPolygon,3763)   |           |          |
 dtmnfr                    | character varying(8)          |           |          |
 freguesia                 | character varying             |           |          |
 municipio                 | character varying             |           |          |
 distrito_ilha             | character varying             |           |          |
 nuts3                     | character varying             |           |          |
 nuts2                     | character varying             |           |          |
 nuts1                     | character varying             |           |          |
 area_ha                   | double precision              |           |          |
 perimetro_km              | integer                       |           |          |
 designacao_simplificada   | character varying             |           |          |
Indexes:
    "cont_freguesias_pkey" PRIMARY KEY, btree (fid)
    "cont_freguesias_geom_geom_idx" gist (geom)
```

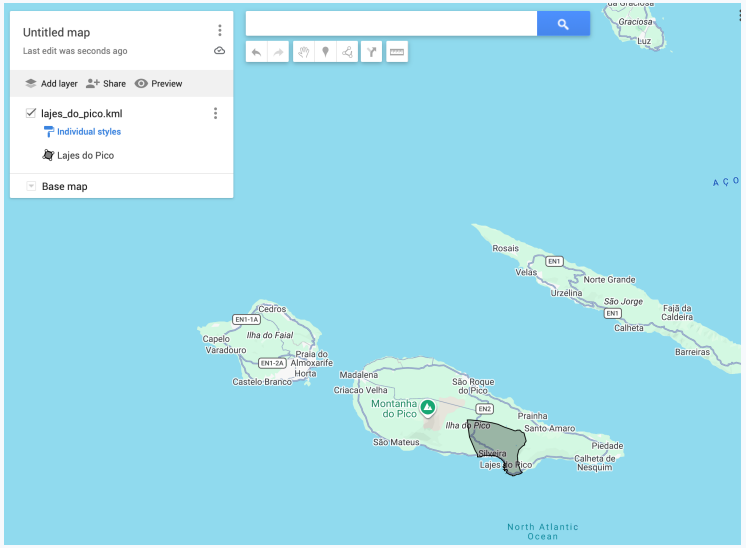
Extracting a parish as KML file and loading it into Google Maps

```
\o /Users/michelferreira/Documents/Aulas/TABD/doc.kml

SELECT '<?xml version="1.0" encoding="UTF-8"?>' ||
       '<kml xmlns="http://www.opengis.net/kml/2.2">' ||
       '<Document>' ||
       '<Placemark><name>' || freguesia || '</name>' ||
       ST_AsKML(ST_Transform(geom, 4326), 8) ||
       '</Placemark>' ||
       '</Document>' ||
       '</kml>'
FROM raa_cen_ori_freguesias
WHERE freguesia = 'Lajes do Pico';

\o
```

- Open https://mymaps.google.com
- Create a new map
- Import the KML file into a layer



Simplifying geometric objects with ST_Simplify

- ST_Simplify returns a simplified version of a geometry using the Douglas-Peucker algorithm.
- Its tolerance is a distance in the units of the input SRS, and for polygons the result may lose simplicity or even validity; when you need topology preserved, PostGIS recommends ST_SimplifyPreserveTopology.

- Example:

  ST_Simplify on the polygon of *Lajes do Pico*
  Reduce the number of vertices to obtain a lighter geometry for display, storage, or web export, at the cost of geometric detail. The larger the tolerance, the stronger the simplification.

```sql
SELECT
    freguesia,
    ST_NPoints(geom) AS original_points,
    ST_NPoints(ST_Simplify(geom, 50))  AS points_50,
    ST_NPoints(ST_Simplify(geom, 100)) AS points_100,
    ST_NPoints(ST_Simplify(geom, 250)) AS points_250
FROM raa_cen_ori_freguesias
WHERE freguesia = 'Lajes do Pico';
```

```
   freguesia    | original_points | points_50 | points_100 | points_250
----------------+-----------------+-----------+------------+------------
 Lajes do Pico |            2436 |        75 |         42 |         21
(1 row)
```

Simplifying geometric objects with ST_Simplify

Aggregate Spatial Functions

- ST_Union works as an aggregate over the rows in each group.
- We can create the aggregates of districts based on the geometric union of all the parishes
  of the same district (for mainland Portugal).
- At this level of detail, it would make sense to use ST_Simplify with a high value of tolerance.
- PostGIS also warns that ST_Simplify may produce invalid output for polygons, so for a
  production-safe version you might prefer ST_SimplifyPreserveTopology.

```
SELECT
    distrito_ilha,
    ST_SimplifyPreserveTopology(ST_Union(geom), 250) AS geom
FROM cont_freguesias
GROUP BY distrito_ilha
ORDER BY distrito_ilha;
```

Two more tables in the Dataset: tracks_braga and taxi_stands

```
                        Table "public.tracks_braga"
   Column   |            Type             | Collation | Nullable | Default
------------+-----------------------------+-----------+----------+---------
 id         | integer                     |           | not null |
 ts         | integer                     |           |          |
 taxi       | character(8)                |           |          |
 state      | character varying(6)        |           |          |
 track      | geometry(LineString,4326)   |           |          |
 proj_track | geometry(LineString,3763)   |           |          |
Indexes:
    "tracks_braga_pkey" PRIMARY KEY, btree (id)
```

```
                        Table "public.taxi_stands"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 id            | integer                  |           | not null |
 name          | character varying(255)   |           |          |
 location      | geometry(Point,4326)     |           |          |
 proj_location | geometry(Point,3763)     |           |          |
Indexes:
    "taxi_stands_pkey" PRIMARY KEY, btree (id)
```

Origin/Destination (O/D) matrix

- An **O/D matrix** summarizes movements between spatial zones.
- Each **row** is an origin zone.
- Each **column** is a destination zone.
- Each **cell** contains the number of trips from one zone to another.

Small example

	Destination		
Origin	Braga	Guimarães	Vila Verde
Braga	120	35	18
Guimarães	22	90	11
Vila Verde	15	27	44

Bottom line

- An O/D matrix converts many individual trips into an aggregated view of mobility flows.

Building an O/D matrix from mobility data

- Define the spatial zones (e.g. municipalities, districts, parishes)
- Identify the **origin** of each trip
- Identify the **destination** of each trip
- Assign both points to zones
- Count trips for each origin-destination pair
- For each trip, we only need to know where it starts and where it ends.

Example: O/D matrix from taxi trajectories

- Table tracks_braga stores taxi trajectories as LineString
- track is in EPSG:4326
- proj_track is in EPSG:3763
- We extract:
  - the **origin** with ST_StartPoint(proj_track)
  - the **destination** with ST_EndPoint(proj_track)

Key idea

Each trajectory becomes one trip from an origin municipality to a destination municipality.

From trajectories to municipality-to-municipality flows

- cont_freguesias gives parish polygons
- We merge parishes into municipality polygons with ST_Union
- Then we locate:
  - the trip start point inside one municipality
  - the trip end point inside another municipality
- Finally, we count trips by municipality pair

```sql
WITH municipalities AS (
    SELECT municipio, ST_Union(geom) AS geom
    FROM cont_freguesias
    GROUP BY municipio
)
SELECT
    mo.municipio AS origin,
    md.municipio AS destination,
    COUNT(*) AS trips
FROM tracks_braga t
JOIN municipalities mo
  ON ST_Covers(mo.geom, ST_StartPoint(t.proj_track))
JOIN municipalities md
  ON ST_Covers(md.geom, ST_EndPoint(t.proj_track))
GROUP BY mo.municipio, md.municipio
ORDER BY mo.municipio, md.municipio;
```

O/D matrix from tracks_braga at municipality level

origin	destination	trips
Amares	Amares	6
Amares	Braga	10
Barcelos	Braga	2
Braga	Amares	9
Braga	Barcelos	3
Braga	Braga	4016
Braga	Guimarães	1
Braga	Maia	7
Braga	Porto	2
Braga	Póvoa de Lanhoso	3
Braga	Terras de Bouro	1
Braga	Vila Nova de Famalicão	6
Braga	Vila Verde	10
Guimarães	Braga	1
Maia	Braga	7
Maia	Maia	1
Paços de Ferreira	Braga	1
Ponte da Barca	Braga	1
Porto	Braga	2
Póvoa de Lanhoso	Braga	2
Póvoa de Lanhoso	Póvoa de Lanhoso	1
Santo Tirso	Braga	1
Terras de Bouro	Braga	1
Vila Nova de Famalicão	Braga	5
Vila Nova de Famalicão	Paços de Ferreira	1
Vila Nova de Famalicão	Vila Nova de Famalicão	4
Vila Verde	Braga	12
Vila Verde	Vila Verde	4

(28 rows)

Integrating PostGIS and PL/pgSQL

- This exercise combines concepts from what we have seen so far:
  - from **Spatial Databases / PostGIS**:
    - trajectories as LineString
    - projected geometries in EPSG:3763
    - ST_NPoints, ST_PointN, ST_Distance
  - from **PL/pgSQL**:
    - creation of user-defined functions
    - variables and loops
    - row-by-row computation with RETURN NEXT

Exercise goal

- Given a taxi trajectory with one sampled point per second, create a PL/pgSQL function that iterates through consecutive points and derives the **speed profile** of the trip.

Key idea

- We use spatial functions to extract and measure the geometry, and PL/pgSQL to control the iterative logic.

Closing line

- This is a good example of how procedural code and spatial SQL can be combined to solve mobility-analysis problems inside PostgreSQL.

Speed Profile of a Track

```
CREATE OR REPLACE FUNCTION track_speed_profile(p_track_id integer)
RETURNS TABLE (
    track_id    integer,
    seq         integer,
    rel_second  integer,
    abs_ts      integer,
    segment_m   double precision,
    speed_mps   double precision,
    speed_kmh   double precision
)
LANGUAGE plpgsql
AS $$
DECLARE
    g       geometry(LineString,3763);
    t0      integer;
    n       integer;
    i       integer;
    p_prev  geometry(Point,3763);
    p_curr  geometry(Point,3763);
    d       double precision;
BEGIN
    SELECT proj_track, ts
    INTO g, t0
    FROM tracks_braga
    WHERE id = p_track_id;

    IF g IS NULL THEN
        RAISE EXCEPTION 'Track % not found, or proj_track is NULL', p_track_id;
    END IF;

    n := ST_NPoints(g);

    IF n < 2 THEN
        RETURN;
    END IF;

    FOR i IN 2..n LOOP
        p_prev := ST_PointN(g, i - 1);
        p_curr := ST_PointN(g, i);

        d := ST_Distance(p_prev, p_curr);

        track_id   := p_track_id;
        seq        := i - 1;
        rel_second := i - 1;

        -- assumes ts is the timestamp of the first point
        IF t0 IS NULL THEN
            abs_ts := NULL;
        ELSE
            abs_ts := t0 + (i - 1);
        END IF;

        segment_m := d;
        speed_mps := d;
        speed_kmh := d * 3.6;

        RETURN NEXT;
    END LOOP;

    RETURN;
END;
$$;
```

Speed Profile of a Track

```sql
SELECT *
FROM track_speed_profile(1000)
ORDER BY seq;
```

track_id	seq	rel_second	abs_ts	segment_m	speed_mps	speed_kmh
1000	1	1	1570668714	4.167166515189624	4.167166515189624	15.001799454682647
1000	2	2	1570668715	4.7155837405528995	4.7155837405528995	16.97610146599044
1000	3	3	1570668716	5.338540844809182	5.338540844809182	19.218747041313055
1000	4	4	1570668717	5.839534682762913	5.839534682762913	21.02232485794649
1000	5	5	1570668718	8.982993503489526	8.982993503489526	32.33877661256229
1000	6	6	1570668719	7.459290773204203	7.459290773204203	26.85344678353513
1000	7	7	1570668720	9.431170567792122	9.431170567792122	33.952214044051644
1000	8	8	1570668721	9.491159802975465	9.491159802975465	34.16817529071167
1000	9	9	1570668722	9.040520839519603	9.040520839519603	32.54587502227057
1000	10	10	1570668723	12.4708905833329	12.4708905833329	44.89520609999844
1000	11	11	1570668724	11.231156958318582	11.231156958318582	40.432165049946896
1000	12	12	1570668725	12.24563896746197	12.24563896746197	44.08430028286309
1000	13	13	1570668726	13.353926580159014	13.353926580159014	48.07413568857245
1000	14	14	1570668727	13.353926399777379	13.353926399777379	48.074135039198566
1000	15	15	1570668728	15.549150375711182	15.549150375711182	55.97694135256026
1000	16	16	1570668729	14.438496439854102	14.438496439854102	51.97858718347477
1000	17	17	1570668730	14.462576429908102	14.462576429908102	52.06527514766917
1000	18	18	1570668731	14.534578073073108	14.534578073073108	52.32448106306319
1000	19	19	1570668732	15.574717498016746	15.574717498016746	56.068982992860285
1000	20	20	1570668733	16.675301334939768	16.675301334939768	60.031084805783166
1000	21	21	1570668734	16.18154122688126	16.18154122688126	58.253548416772546
1000	22	22	1570668735	16.0158887931055	16.0158887931055	57.6571996551798
1000	23	23	1570668736	18.01355628558763	18.01355628558763	64.84880262811546
1000	24	24	1570668737	20.043474897464318	20.043474897464318	72.15650963087154
1000	25	25	1570668738	20.898933519380208	20.898933519380208	75.23616066976875
1000	26	26	1570668739	20.74312500215718	20.74312500215718	74.67525000776584
1000	27	27	1570668740	20.898980972586138	20.898980972586138	75.2363315013101
1000	28	28	1570668741	21.63532091751329	21.63532091751329	77.88715530304785
1000	29	29	1570668742	22.378537512317624	22.378537512317624	80.56273504434346
1000	30	30	1570668743	22.896235298865708	22.896235298865708	82.42644707591656
1000	31	31	1570668744	22.896262440895708	22.896262440895708	82.42654478722454
1000	32	32	1570668745	24.894634092071534	24.894634092071534	89.62068273145752

```sql
SELECT
    track_id,
    AVG(speed_kmh) AS avg_speed_kmh,
    MAX(speed_kmh) AS max_speed_kmh,
    SUM(segment_m) / 1000.0 AS total_km
FROM track_speed_profile(1000)
GROUP BY track_id;
```

track_id	avg_speed_kmh	max_speed_kmh	total_km
1000	27.478756624046465	91.6499117555176	2.7249766985512727

(1 row)